```
%macro io 4                                     ; Macro for syscall with four parameters
        mov rax,%1                              ; System call number
        mov rdi,%2                              ; First argument (file descriptor)
        mov rsi,%3                              ; Second argument (buffer)
        mov rdx,%4                              ; Third argument (size)
        syscall                                 ; Invoke system call
%endmacro

%macro exit 0                                   ; Macro to exit the program
        mov rax,60                              ; syscall number for exit
        mov rdi,0                               ; Exit code 0
        syscall                                 ; Invoke system call
%endmacro

section .data                                   ; Data section
   msg5 db "write an X86/64 ALP to accept a string and to display its length",
   10, 'Name:-Sumaanyu', 10, 'roll:-7256 ',
   10 ,'Date Of Performance:-20/01/2025',
   10                                           ; Instruction and metadata message
   msg5len equ $-msg5                           ; Length of msg5
        msg1 db "Enter some string:",20H        ; Prompt message for user input
        msg1len equ $-msg1                      ; Length of msg1
        msg2 db "The length is: ", 20H          ; Message to display string length
        msg2len equ $-msg2                      ; Length of msg2
   msg3 db "step1: ", 10                        ; Step 1 message
        msg3len equ $-msg3                      ; Length of msg3
   msg4 db "step2: ", 10                        ; Step 2 message
        msg4len equ $-msg4                      ; Length of msg4
   newline db 10                                ; Newline character

section .bss                                    ; Uninitialized data section
        strna resb 100                          ; Buffer to store user input string
        len1 resb 1                             ; Variable to store string length
        len2 resb 1                             ; Secondary length counter
   lenca resb 2                                 ; Buffer to store ASCII hex representation

section .text                                   ; Code section
        global _start                           ; Entry point
        _start:
     io 1,1,msg5,msg5len                        ; Print msg5 to stdout
                io 1,1,msg1,msg1len             ; Print msg1 (prompt for input)
     io 0,0,strna,20                            ; Read user input into strna (max 20 chars)
                dec rax                         ; Adjust length to exclude newline
                mov [len1],rax                  ; Store length in len1
     io 1,1,msg3,msg3len                        ; Print step1 message
     io 1,1,msg2,msg2len                        ; Print "The length is: "
     mov bl,[len1]                              ; Move length value into bl
     call hex_ascii64                           ; Convert to hex and print
     io 1,1,msg4,msg4len                        ; Print step2 message
     mov rcx,[len1]                             ; Load string length into rcx
     next1:
        mov rsi,strna                           ; Load string address into rsi
```

```
        mov al,[rsi]                    ; Get first character
        cmp al,10                       ; Compare with newline
        jne inby                        ; If not newline, continue
        inby:
            inc byte[len2]              ; Increment length counter
            loop next1                  ; Repeat until end of string
    io 1,1,msg2,msg2len                 ; Print "The length is: " again
    mov bl,[len2]                       ; Move updated length into bl
    call hex_ascii64                    ; Convert and print
                        exit            ; Exit the program
hex_ascii64:
    mov rsi,lenca                       ; Load ASCII buffer
    mov rcx,2                           ; Loop count (2 hex digits)
    next2:
        rol bl,4                        ; Rotate left by 4 bits
        mov al,bl                       ; Copy to al
        and al,0fh                      ; Mask lower nibble
        cmp al,9                        ; Compare with 9
        jbe add30h                      ; If <= 9, add 30h
        add al,7H                       ; Adjust for A-F characters
        add30h:
            add al,30H                  ; Convert to ASCII digit
            mov [rsi],al                ; Store in buffer
            inc rsi                     ; Move to next buffer position
    loop next2
        io 1,1,lenca,2                  ; Print hex representation
        io 1,1,newline,1                ; Print newline
    ret                                 ; Return from function
```